

FAST

FORMAÇÃO ACELERADA EM
SOLUÇÕES DE TECHDESIGN



C.E.S.A.R.

Automação de Testes Web

Todos os direitos reservados.

Este material, ou qualquer parte dele, não pode ser reproduzido,
divulgado ou usado de forma alguma sem autorização escrita.



Testes de Automação Web

Automação Web
Aula 4



Estrutura do curso

Conteúdo:

Aula 1:

- Fundamentos da Automação de Testes
- Diferenças entre testes manuais e automatizados
- Estrutura de um Script de Teste
- Ferramentas

Aula 2:

- Identificação de Elementos
- Elementos dinâmicos e boas práticas
- Introdução ao padrão page object

Aula 3:

- Organização com page object
- Comandos, esperas e interações básicas
- Comandos, esperas e validações

Aula 4:

- Estrutura de projeto e boas práticas
- Modularização e reuso de código
- Construção de suite de testes

Aula 5:

- Criação de fluxo de teste completo
- Prática guiada
- Bonus: CI/CD + Paralelização

Intervalo: 15 à 20 minutos

Material das aulas: NEO

Avaliação Final: 5 questões objetivas

O que veremos hoje?

- Estrutura de projeto e boas práticas
- Modularização e reuso de código
- Construção de suite de testes

Estrutura de projeto e boas práticas

Estrutura recomendada de projeto (Maven + Java + POM)

bash

 Copy  Edit

meu-projeto-automacao/

├── src/

│ ├── main/

│ │ └── java/

│ │ └── utils/

│ │ # Utilitários diversos (esperas, leitura de config,

│ └── test/

│ ├── java/

│ │ ├── tests/

│ │ # Classes de teste (JUnit/TestNG)

│ │ ├── pages/

│ │ # Page Objects com os mapeamentos e métodos de inter

│ │ └── runners/

│ │ # Runners de execução (em caso de Cucumber)

│ └── resources/

│ ├── data/

│ # Massa de dados (CSV, JSON, XML etc.)

│ └── config/

│ # Arquivos de configuração (por exemplo, 'config.pro

└── pom.xml

Arquivo do Maven (gerência de dependências e plugi

└── README.md

Estrutura de projeto e boas práticas

Explicação das Camadas

Camada	Responsabilidade	
tests	Contém os testes propriamente ditos (classes JUnit/TestNG com <code>@Test</code>)	
pages	Representa as páginas da aplicação. Contém os elementos mapeados e métodos de ação. Ex: <code>LoginPage</code> , <code>HomePage</code>	
utils	Métodos auxiliares que não pertencem diretamente a uma página ou teste. Ex: espera explícita, leitura de arquivos, manipulação de datas	
resources	Dados e configurações externas ao código Java. Ideal para deixar parâmetros de ambiente ou massa de testes	
pom.xml	Define as dependências, plugins e configurações do Maven	

Estrutura de projeto e boas práticas

✓ Boas práticas de estrutura

- 1 classe de Page para cada página da aplicação (ex: `LoginPage`, `DashboardPage`)
- Separar os testes das interações: os testes chamam métodos das pages, nunca fazem `findElement` diretamente
- Uso de herança para WebDriver: uma `BaseTest` com setup e teardown para evitar duplicação
- Utilizar o padrão Page Factory (opcional) para inicialização mais limpa dos elementos
- Nomear classes e métodos de forma expressiva:
`loginComUsuarioValido_deveRedirecionarParaDashboard()`

Modularização e reuso de código

1. Utilize o padrão Page Object

- Cada página da aplicação deve ter sua própria classe.
- Centraliza os elementos e interações em um só lugar.

```
java

// LoginPage.java
public class LoginPage {
    @FindBy(id = "username") WebElement campoUsuario;
    @FindBy(id = "password") WebElement campoSenha;
    @FindBy(id = "loginButton") WebElement botaoLogin;

    public void preencherUsuario(String usuario) {
        campoUsuario.sendKeys(usuario);
    }

    public void preencherSenha(String senha) {
        campoSenha.sendKeys(senha);
    }

    public void clicarLogin() {
        botaoLogin.click();
    }
}
```

Modularização e reuso de código

2. Crie métodos utilitários (helpers)

- Para tarefas comuns como esperar elemento, tirar print, limpar cookies etc.

```
java                                                                    Copy Edit

public class WaitHelper {
    WebDriver driver;
    WebDriverWait wait;

    public WaitHelper(WebDriver driver) {
        this.driver = driver;
        this.wait = new WebDriverWait(driver, Duration.ofSeconds(10));
    }

    public WebElement esperarVisibilidade(By locator) {
        return wait.until(ExpectedConditions.visibilityOfElementLocated(locator));
    }
}
```

Modularização e reuso de código

3. Centralize dados de teste

- Use arquivos `.properties`, `.json`, `.yaml` ou tabelas no `Scenario Outline` para evitar valores hardcoded.

gherkin

 Copy  Edit

Examples:

usuario	senha	
admin	123456	

Modularização e reuso de código

4. Reutilize Steps genéricos

- Escreva definições de passos reutilizáveis com expressões genéricas.

```
java
```

```
@When("informa o usuário {string} e a senha {string}")  
public void informarCredenciais(String usuario, String senha) {  
    loginPage.preencherUsuario(usuario);  
    loginPage.preencherSenha(senha);  
}
```

Modularização e reuso de código

5. Use Hooks para ações comuns

- Como iniciar/fechar browser, limpar dados, capturar evidência, login comum, etc.

java

 Copy  Edit

```
@Before
public void iniciarTeste() {
    driver = WebDriverFactory.createDriver();
    loginPage = new LoginPage(driver);
}

@After
public void finalizarTeste(Scenario scenario) {
    if (scenario.isFailed()) {
        ScreenshotHelper.capture(driver, scenario.getName());
    }
    driver.quit();
}
```

Modularização e reuso de código

6. Evite repetição de código

- Sempre que encontrar 2+ repetições, crie um método.

Exemplo: cliques semelhantes, esperas padrão, scrolls etc.

java

```
public class LoginPage {
    private WebDriver driver;

    public LoginPage(WebDriver driver) {
        this.driver = driver;
    }

    private By usuario = By.id("username");
    private By senha = By.id("password");
    private By botaoLogin = By.id("loginButton");

    public void fazerLogin(String user, String pass) {
        driver.findElement(usuario).sendKeys(user);
        driver.findElement(senha).sendKeys(pass);
        driver.findElement(botaoLogin).click();
    }
}
```

java

Copy Edit

```
@Test
public void testLoginValido() {
    LoginPage login = new LoginPage(driver);
    login.fazerLogin("admin", "123");
    Assert.assertEquals("https://exemplo.com/dashboard", driver.getCurrentUrl());
}

@Test
public void testLoginInvalido() {
    LoginPage login = new LoginPage(driver);
    login.fazerLogin("admin", "errado");
    WebElement erro = driver.findElement(By.id("erro"));
    Assert.assertTrue(erro.isDisplayed());
}
```

Modularização e reuso de código

7. Crie classes base

- Uma `BasePage` e `BaseTest` para abstrair código comum.

java

 Copy  Edit

```
public class BasePage {  
    protected WebDriver driver;  
    protected WebDriverWait wait;  
  
    public BasePage(WebDriver driver) {  
        this.driver = driver;  
        this.wait = new WebDriverWait(driver, Duration.ofSeconds(10));  
    }  
  
    protected void clicar(WebElement el) {  
        wait.until(ExpectedConditions.elementToBeClickable(el)).click();  
    }  
}
```

Modularização e reuso de código

8. Organize a estrutura de pastas

```
bash
```

```
src/test/java/
```

```
|— features/           # Arquivos .feature  
|— steps/             # Definições de passos  
|— pages/             # Page Objects  
|— support/           # DriverFactory, Hooks, Helpers  
|— runners/           # Runners de Cucumber  
└─ data/              # Massa de dados (JSON/YAML/etc)
```

Modularização e reuso de código

9. Separe testes por contexto

- Se possível, crie diferentes páginas e pastas por módulo: `LoginPage`, `CadastroPage`, `ProdutoPage`, etc.
-

10. Evite lógica de negócios nos testes

- Validações complexas devem ser feitas em métodos da Page ou Helpers. Os testes só devem chamar os métodos.

DÚVIDAS?!
:)



INTERVALO!



20 min.

Construção de suite de testes

Estrutura do projeto com Cucumber integrado

```
bash Copy Edit

neu-projeto-automacao/
├── src/
│   ├── main/
│   │   └── java/
│   │       └── utils/           # Utilitários (esperas, helpers, propriedades)
│   └── test/
│       ├── java/
│       │   ├── steps/         # Step Definitions (implementação dos cenários Ghe
│       │   ├── pages/         # Page Objects
│       │   ├── runners/       # Runner Cucumber + JUnit
│       │   └── support/       # Hooks (Before, After), base de testes
│       └── resources/
│           ├── features/      # Arquivos .feature (cenários Gherkin)
│           ├── data/          # Massa de dados (opcional)
│           └── config/        # Arquivos de configuração
├── pom.xml
└── README.md
```

Construção de suite de testes

✓ Boas Práticas do Cucumber

1. Escreva cenários em linguagem de negócio

- Evite termos técnicos de automação (como `click`, `findElement`, `xpath`) no `.feature`
- Use uma linguagem que qualquer pessoa da equipe (QA, PO, Dev) entenda.

Ruim:

gherkin

📄 Copy ✎ Edit

```
When eu clico no botão com ID "btnLogin"
```

Bom:

gherkin

📄 Copy ✎ Edit

```
When eu clico no botão de login
```

Construção de suite de testes

✓ Boas Práticas do Cucumber

2. Use um cenário por comportamento

- Prefira cenários pequenos, que validem um comportamento específico.

Ruim:

```
gherkin
```

```
Scenario: Login, navegação e logout
```

Bom:

```
gherkin
```

```
Scenario: Login com sucesso
```

```
Scenario: Logout do sistema
```

Construção de suite de testes

✓ Boas Práticas do Cucumber

3. Evite duplicação com *Background*

- Use `Background` para ações comuns a todos os cenários.

```
gherkin
```

```
Background:
```

```
  Given que o usuário está logado no sistema
```

```
Scenario: Visualizar perfil
```

```
  When acessa a aba de perfil
```

```
  Then deve ver suas informações
```

Construção de suite de testes

✓ Boas Práticas do Cucumber

4. Use Step Definitions reutilizáveis

- Evite criar múltiplos métodos Java para passos semelhantes.
- Use expressões regulares com parâmetros ({string}, {int}, etc.)

gherkin

```
When eu preencho o campo "usuário" com "admin"
```

java

```
@When("eu preencho o campo {string} com {string}")  
public void preencherCampo(String campo, String valor) {  
    loginPage.preencherCampo(campo, valor);  
}
```

Construção de suite de testes

✓ Boas Práticas do Cucumber

5. Organize o projeto em camadas

- Separe bem as responsabilidades:
 - `.feature` : linguagem de negócio
 - `steps/` : lógica dos testes
 - `pages/` : interação com elementos da interface
 - `support/` : driver, hooks, helpers, configuração

Construção de suite de testes

✓ Boas Práticas do Cucumber

6. Não coloque lógica de automação no `.feature`

- O `.feature` deve ser uma especificação, não um roteiro técnico.

Ruim:

```
gherkin
```

```
Then espero 5 segundos e clico no botão com ID "btnX"
```

Bom:

```
gherkin
```

```
Then o botão de avançar deve estar visível
```

Construção de suite de testes

✓ Boas Práticas do Cucumber

7. Nomeie arquivos e cenários de forma clara

- Use nomes que indiquem exatamente o que está sendo testado.

```
gherkin
```

```
Feature: Login do sistema
```

```
Scenario: Login com credenciais válidas
```

```
Scenario: Login com senha incorreta
```

Construção de suite de testes

✓ Boas Práticas do Cucumber

8. Evite criar muitos cenários para a mesma variação

- Utilize `Scenario Outline` para entradas diferentes com o mesmo fluxo.

gherkin

Scenario Outline: Login com credenciais inválidas

Given que estou na tela de login

When eu preencho o usuário "<usuario>" e a senha "<senha>"

And cliço no botão de login

Then devo ver a mensagem "<mensagem>"

Examples:

usuario	senha	mensagem	
errado1	123	Usuário ou senha inválidos	
admin	errado123	Usuário ou senha inválidos	

Construção de suite de testes

✓ Boas Práticas do Cucumber

9. Use Tags para agrupar e controlar execuções

- Tags permitem executar apenas cenários de um grupo:

```
gherkin
```

```
@regressivo
```

```
Scenario: Login com sucesso
```

```
bash
```

```
mvn test -Dcucumber.filter.tags="@regressivo"
```

Construção de suite de testes

📁 Arquivo: login.feature

gherkin

📄 Copy 📄 Edit

@regressivo @login

Feature: Login no sistema

Como um usuário do sistema

Quero acessar minha conta

Para utilizar as funcionalidades disponíveis

Background:

Given que o usuário está na página de login

Scenario: Login com credenciais válidas

When informa o usuário "admin" e a senha "123456"

And clica no botão de login

Then deve ser redirecionado para a página inicial

Scenario Outline: Login com credenciais inválidas

When informa o usuário "<usuario>" e a senha "<senha>"

And clica no botão de login

Then deve visualizar a mensagem de erro "<mensagem>"

Examples:

usuario	senha	mensagem
admin	errado	Usuário ou senha inválidos
desconhe	123456	Usuário ou senha inválidos
**	**	Campo de usuário e senha obrigatório

DÚVIDAS?!
:)





C . E . S . A . R

MUITO OBRIGADO!

© CESAR | Todos os direitos reservados